

课程笔记

Codex 通用工具设计：subagents、 apply_patch、exec_command 与 agentic vision

五道口纳什

2026 年 4 月 14 日

Codex General Tool sub-agents

视频作者/频道：五道口纳什

发布日期：2025 年 3 月 28 日

视频时长：23 分 11 秒

视频链接：<https://www.bilibili.com/video/BV1QUXGBXER9>

目录

1 引言：从黑盒到白盒的 Agent 设计观	2
1.1 系列定位与本期目标	2
1.2 ”没有魔法”：为什么要拆解 Codex 的 general tool design	3
1.3 底层模型能力 vs runtime / harness 维护的分工边界	4
1.4 本章小结	4
2 Function Calling 与 Tool Spec 的三层架构	5
2.1 内部规格 (Internal Spec)：Codex 自己维护的 ToolSpec 抽象	5
2.2 传输定义 (Wire Definition)：序列化后发给 response.create 的 JSON	5
2.3 执行层 (Handler / Runtime)：本地解析、校验、执行并回填结果	7
2.4 一个关键细节：tool schema 只有入参描述，返回值由 runtime 回填到上下文	7
2.5 本章小结	10
3 apply_patch DSL	10
3.1 为什么不用 freestyle 写文件，而是用 patch DSL	10
3.2 Lark grammar 约束与结构化输出	11
3.3 Handler 的执行流程：语法校验 → 推导变更文件 → 权限审批 → 实际写入	11
3.4 系统提示词中的使用规范	12
3.5 本章小结	12
4 Subagents 系统：并发、隔离与汇总	13
4.1 设计动机：上下文隔离、并发执行与 context window 突破	13
4.2 五个核心工具：spawn、send_input、wait、close、resume	13
4.3 spawn_agent 的返回值机制：agent_id 与 nickname 的回填	14
4.4 wait_for_subagent：状态流订阅而非 sleep	15
4.5 触发策略与当前接口的局限	16
4.6 本章小结	17
5 view_image：Agentic Vision 的本地桥接	18
5.1 为什么需要专门的 view_image 工具，而非直接塞 base64	18
5.2 runtime 职责：路径解析 → 读取文件 → base64 编码 → 注入上下文	19
5.3 使用条件：完整路径 + 历史上未查看过	20
5.4 多模态能力与 coding agent 的结合	21
5.5 本章小结	21
6 总结与延伸	22
6.1 演讲者的核心收束	22
6.2 结构化提炼：五条核心洞察	22
6.3 从 tool design 反推：如何 review 和引导 agent	23
6.4 拓展阅读	23

1 引言：从黑盒到白盒的 Agent 设计观

系列背景

本课程属于 *Modern Agent* 系列的延续。在前面的第 12、13、15 期中，我们分别从 context engineering、runtime 与 harness 三个角度对 OpenAI Codex 进行了系统剖析。本期将把焦点放在 Codex 的 **general tool design** 上——即支撑其 agentic 行为的 12 个预定义工具。

1.1 系列定位与本期目标

Coding agent 的能力边界，往往不取决于底层模型本身，而取决于它如何被“武装”成可执行的结构化操作。Codex 作为当前最具代表性的 coding agent 之一，其内部包含了 12 个预定义工具，覆盖了代码编辑、命令执行、子代理并发、网络搜索、视觉输入等核心场景。本期课程的目标，就是逐个拆解这些工具的设计思想与实现边界，帮助读者建立起对 agent 系统的“白盒”认知。

从内容编排上看，本期将依次覆盖以下主题：

- **Function Calling 与 Tool Spec 的三层架构**：理解 internal spec、wire definition 与 handler/runtime 的分层逻辑；
- **apply_patch**：代码编辑为何采用受限的 patch DSL，而非自由文本重写；
- **exec_command 与 write_stdin**：命令行交互的两种模式，以及“bash is all you need”的工程含义；
- **Subagents 系统**：并发、隔离与汇总的实现机制；
- **view_image**：多模态输入如何通过 runtime 桥接进入模型上下文；
- 其余补充工具（web_search、update_plan、request_user_input 等）的速览与总结。

通过这一整套分析，我们将看到：Codex 的 agentic 行为并非“魔法”，而是一组经过精心设计的工具调用链路与 runtime 回填机制共同作用的结果。

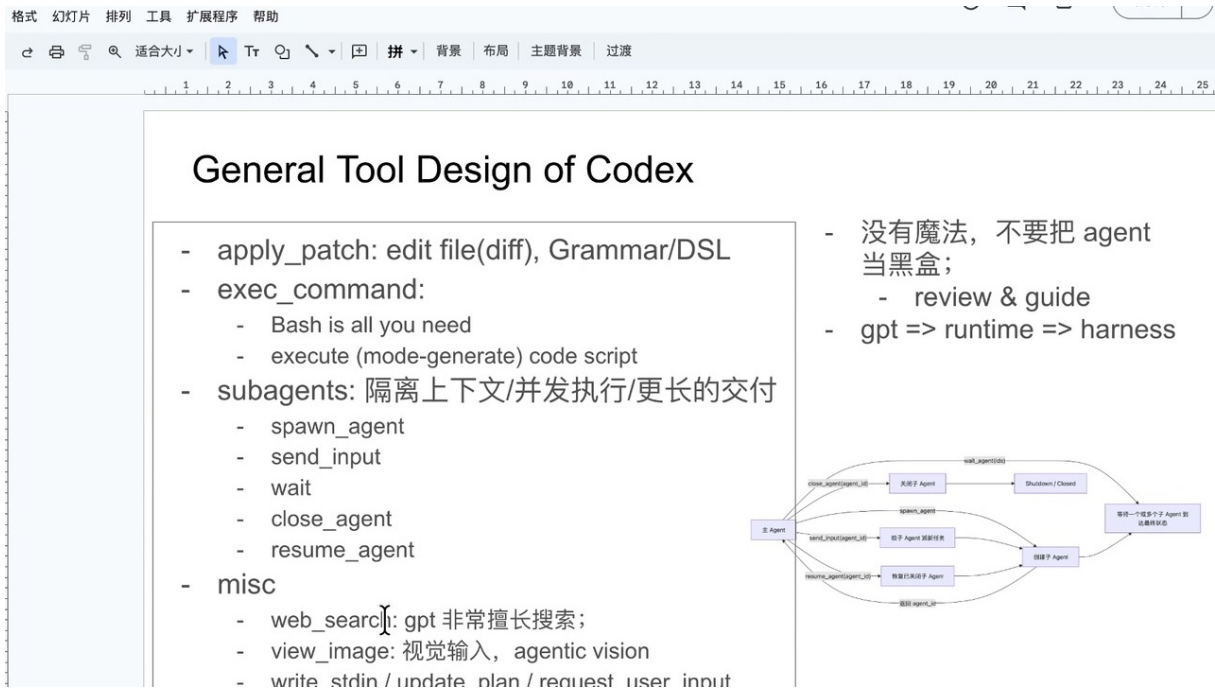


图 1: 开篇总览: Codex 通用工具设计的整体框架, 涵盖 apply_patch、exec_command、subagents、web_search、view_image 等 12 个核心工具。¹

1.2 ”没有魔法”: 为什么要拆解 Codex 的 general tool design

在观察现代 coding agent 时, 一个常见的误区是将其视为黑盒: 当输出符合预期时赞叹其“智能”, 当输出偏离预期时则简单归结为“模型能力不足”。然而, 真正有效的调试与优化策略, 应当是打开这个黑盒, 去 **review** 并引导 **agent** 的交付过程。

核心主张: 没有魔法

Codex 的 general tool design 是理解 coding agent 的关键。当 agent 表现不符合预期时, 应首先 review 其 tool use 过程, 而非盲目重试或归咎于模型能力。

拆解工具设计的价值体现在三个层面:

1. **诊断定位**: 当模型陷入死胡同时, 通过检查它调用了哪些工具、参数是否正确、runtime 是否按预期回填了结果, 可以快速判断问题出在模型决策层还是 runtime 执行层。
2. **能力边界认知**: 理解哪些行为是底层模型本身的能力 (如代码生成、推理、视觉理解), 哪些行为是 runtime / harness 通过工具封装后“嫁接”给模型的, 有助于设定合理的预期。
3. **系统优化方向**: 如果发现是工具描述 (description) 或 schema 设计不当导致模型调用失误, 那么优化方向就不是“换更大的模型”, 而是改进工具定义与系统提示词。

¹ 视频画面时间区间: 00:03:18-00:03:22。

常见误区

不要一遇到 agent 输出不佳就默认是模型能力不足。在很多实际场景中，问题根源在于工具描述模糊、handler 执行异常、或返回值回填格式不符合模型预期。

1.3 底层模型能力 vs runtime / harness 维护的分工边界

在 Codex 的架构中，底层大模型与 runtime / harness 之间存在一条清晰的分工边界。理解这条边界，是后续分析每一个具体工具的前提。

- **底层模型能力**：指模型本身通过预训练和对齐所获得的通用能力，包括自然语言理解、代码生成、逻辑推理、多模态感知等。模型在生成回复时，会根据上下文决定“说什么”以及“调用哪个工具”。
- **runtime / harness 维护**：指在模型之外由系统代码负责的全部执行逻辑，包括工具定义的序列化与下发、function call 参数的解析与校验、本地命令或文件操作的实际执行、以及将执行结果以 `function_call_output` 的形式回填到模型上下文中。

以命令执行为例：模型本身并不具备直接操作本地文件系统或启动进程的能力。它所能做的，只是根据用户请求和系统提示词，生成一段类似 `exec_command` 的 JSON 调用请求。真正创建进程、执行脚本、捕获标准输出、并将输出包装后返回给模型的，是 runtime 中的 handler 代码——这正是第 2 章将详细说明的 function calling 机制。

再如代码编辑：模型输出的是一段符合 patch DSL 语法的文本，但语法校验、目标文件推导、权限审批以及最终的磁盘写入，全部发生在 runtime 层。模型“知道”要修改什么，但“能否安全地修改”由 runtime 把关。

分工边界的工程意义

底层模型负责 **决策与生成**，runtime / harness 负责 **执行与回填**。这种分离使得 Codex 可以在不更换底层模型的情况下，通过扩展工具集合、优化 handler 逻辑、或调整系统提示词来持续提升 agent 能力。

1.4 本章小结

本章作为全课程的引言，确立了以下核心认知：

1. 本期课程聚焦 Codex 的 **12 个预定义工具**，是对 Modern Agent 系列在 context engineering、runtime、harness 分析基础上的自然延伸。
2. 分析这些工具的目的，是为了建立“**没有魔法**”的白盒视角——当 agent 行为异常时，应优先 review 其 tool use 过程，而非简单归因于模型能力。
3. Codex 的能力来自 **底层模型与 runtime / harness** 的协同：前者负责决策与生成，后者负责执行、校验与结果回填。后续各章对每一个工具的剖析，都将围绕这一分工边界展开。

下一章，我们将进入工具设计的抽象层，系统讲解 Codex 中 **Function Calling 与 Tool Spec** 的三层架构，为后续逐工具分析奠定共同的理论框架。

2 Function Calling 与 Tool Spec 的三层架构

在深入分析 Codex 的 12 个预定义工具之前，我们需要先建立一个统一的认知框架：Codex 中的工具定义并非简单的 JSON 列表，而是存在明显的三层分离——内部规格（internal spec）、传输定义（wire definition）与执行层（handler / runtime）。理解这三层之间的边界与交互方式，是后续解读每个具体工具设计意图的前提。

前置知识：Function Calling 的基本语义

Function calling（函数调用）是指大模型在生成回复时，输出结构化的工具调用请求（包含工具名与参数），由运行时在本地执行该工具，并将执行结果以特定消息格式回填到模型的后续上下文中。模型本身并不直接执行任何代码或命令，所有的实际副作用都发生在 runtime 层。

2.1 内部规格（Internal Spec）：Codex 自己维护的 ToolSpec 抽象

Codex 的源码（主要在 `spec.rs` 中）维护了一套完整的内部规格层，称为 ToolSpec。这一层是 Codex 自己对工具的认知模型，包含以下核心字段：

- **name**：工具的唯一标识名，例如 `spawn_agent`、`apply_patch`。
- **description**：供模型阅读的自然语言描述，决定模型在何种场景下会选择调用该工具。
- **parameters / input schema**：约束模型生成参数时的 JSON Schema，仅描述入参结构。
- **output_schema**：描述该工具理论上的返回结构，用于代码层面的类型约束与文档说明。

以 `spawn_agent` 为例，其内部规格不仅说明了入参（如 `prompt`、`model` 等），还定义了 `output_schema`，指出执行成功后理论上会返回 `agent_id` 和 `nickname`。然而，这一 `output_schema` 仅存在于内部规格层，并不会出现在后续发给模型的请求体中。这是三层架构中最容易被忽视、却也最关键的一个设计细节。

2.2 传输定义（Wire Definition）：序列化后发给 `response.create` 的 JSON

第二层是 wire definition，即实际通过网络传输、塞入 `response.create` 请求体中的工具定义。Codex 在调用 OpenAI Responses API 之前，会将内部规格序列化为符合 API 要求的 JSON 格式。这个序列化过程会**主动跳过** `output_schema`。

讲师在视频中展示了一个完整的请求体落盘文件（可通过环境变量 `OPENAI_CODEX_LOG_RESPONSES=1` 开启落盘），其中包含了 `instructions`、`input` 消息列表，以及 12 个工具的完整配置。下面是一个经过简化的请求体片段，展示了工具列表在请求体中的位置：

```
1 {
2   "instructions": "...",
3   "input": [
4     { "role": "user", "content": { "type": "input_text", "text": "..." } }
5   ],
6   "tools": [
7     { "type": "function", "name": "apply_patch", "description": "...", "parameters"
8       : { ... } },
9     { "type": "function", "name": "exec_command", "description": "...", "parameters"
10       : { ... } },
11     ...
12   ]
13 }
```

```

9      { "type": "function", "name": "write_stdin", "description": "...", "parameters"
      : { ... } },
10     { "type": "function", "name": "spawn_agent", "description": "...", "parameters"
      : { ... } },
11     { "type": "function", "name": "send_input", "description": "...", "parameters":
      { ... } },
12     { "type": "function", "name": "wait_for_subagent", "description": "...", "
      parameters": { ... } },
13     { "type": "function", "name": "close_subagent", "description": "...", "
      parameters": { ... } },
14     { "type": "function", "name": "resume_subagent", "description": "...", "
      parameters": { ... } },
15     { "type": "function", "name": "view_image", "description": "...", "parameters":
      { ... } },
16     { "type": "function", "name": "web_search", "description": "...", "parameters":
      { ... } },
17     { "type": "function", "name": "update_plan", "description": "...", "parameters"
      : { ... } },
18     { "type": "function", "name": "request_user_input", "description": "...", "
      parameters": { ... } }
19 ]
20 }

```

Listing 1: `response.create` 请求体中的 `tools` 字段（简化示意）

注意，上述每个工具对象中只有 `name`、`description` 和 `parameters`，没有任何关于返回值的声明。这与 OpenAI Function Calling 官方文档的约定完全一致：tool schema 只约束入参，不声明出参。

```

s_response_id "resp_0e9fe8be55013b400169c50b628ea48191a6f647bc33556a3f"
] 3 items

> "message"
  "developer"
tent [] 3 items

> "message"
  "user"
tent [] 1 item

> "message"
  "user"
tent [] 1 item
] 12 items
choice "auto"
_tool_calls true

```

图 2: `responses.create` 实际请求体结构截图，展示 `instructions`、`input` 消息列表以及 12 个 `tools` 的完整配置。²

2.3 执行层 (Handler / Runtime): 本地解析、校验、执行并回填结果

当模型根据 wire definition 中的描述决定调用某个工具后, 它会生成一段 `function_call` 消息, 包含工具名和参数。Codex 的 runtime 层接收到这段消息后, 会进入对应的 handler 执行以下流程:

1. **解析参数**: 从模型生成的 JSON 中提取参数值。
2. **校验与合并**: 检查参数类型、必填项, 以及是否符合额外的业务约束。
3. **本地执行**: 调用实际的系统能力 (如写文件、启动进程、创建子代理、读取图片等)。
4. **构造结果**: 将执行产出封装为 `function_call_output`。
5. **回填上下文**: 把 `function_call_output` 注入到后续 `response.create` 的 `input` 消息列表中。

整个执行过程发生在用户本地机器上, 而非远端模型服务器。模型对执行结果的全部感知, 都依赖于 runtime 在第 5 步中的回填动作。这一“解析 → 执行 → 回填”的闭环, 正是所有 Codex 工具共享的通用 handler 模式。



图 3: 执行实现层与结果回填层: 模型发起 `spawn_agent` 调用后, runtime 进入 handler 解析参数、执行并构造 `SpawnAgentResult`, 最终包装为 `function_call_output` 回填上下文。³

2.4 一个关键细节: tool schema 只有人参描述, 返回值由 runtime 回填到上下文

这是理解 Codex 工具设计的核心洞察。让我们再次以 `spawn_agent` 为例:

- 在内部规格层, `spawn_agent` 的 `output_schema` 声明了返回值包含 `agent_id` 和 `nickname`。
- 在传输定义层, 发给模型的 JSON 中完全没有这些返回值的影子。

²视频画面时间区间: 00:07:54–00:07:58。若为裁剪图, 时间区间仍对应原视频画面。

³视频画面时间区间: 00:10:44–00:10:48。若为裁剪图, 时间区间仍对应原视频画面。

- 在执行层，runtime 创建完子代理后，会主动生成一个 `function_call_output`，把 `agent_id` 和 `nickname` 塞进去。

模型正是通过这个被回填的 `function_call_output`，才“知道”自己刚才创建了一个什么样的子代理，从而能在后续对话中使用 `send_input` 或 `wait_for_subagent` 等工具与之交互。

核心结论：output_schema 不发模型，模型通过回填感知返回值

`output_schema` 仅存在于 Codex 的内部规格层，用于代码维护和文档说明；它不会出现在发给 `response.create` 的 wire definition 中。模型对工具返回值的全部认知，都来自于 runtime 以 `function_call_output` 形式回填到上下文中的结果。这是 internal spec、wire definition 和 handler 三层之间最关键的分工边界。

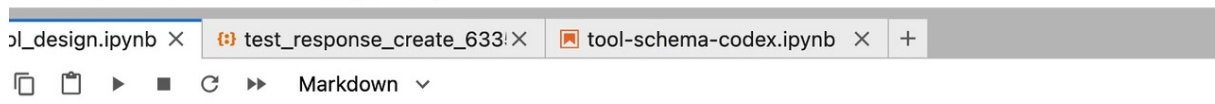
常见误解

初学者容易误以为：既然内部规格有 `output_schema`，那么模型在调用前就已经“知道”工具会返回什么结构。事实上，模型对返回值的感知是**后置的**——只有在 runtime 执行并回填之后，模型才能看到实际的返回内容。模型在生成调用请求时，只依据 `description` 和 `parameters` 做决策。

下面是一个 `spawn_agent` 执行后，runtime 回填到上下文的 `function_call_output` 示例：

```
1 {  
2   "type": "function_call_output",  
3   "call_id": "call_xxx",  
4   "output": "{ \"agent_id\": \"agent_abc123\", \"nickname\": \"Explorer\" }"  
5 }
```

Listing 2: `spawn_agent` 的 `function_call_output` 示例



- <https://github.com/openai/codex/blob/main/codex-rs/core/src/tools/spec.rs>
- 内部规格层: Codex 自己维护的 ToolSpec
 - 一层定义的是 Codex 内部的工具抽象, 而不是直接发给 responses.create 的 JSON。在 create_spawn_agent_tool Codex 构造了一个 ToolSpec::Function(ResponsesApiTool), 里面包含:
 - name
 - description
 - parameters
 - output_schema
 - 其中 spawn_agent_output_schema() 只是这个内部 ToolSpec 的一个字段, 用来描述该工具“理论上的输出结构”; 它独的一层, 也不是最终直接发给模型的 tool definition。
- wire tool definition: 真正发给 responses.create.tools (wire 电线/网线, 工具在网络请求中的传输定义)
 - 这一层才是 wire layer。Codex 会把内部的 ToolSpec 通过 create_tools_json_for_responses_api() 序列化化成 JSON 进 ResponsesApiRequest.tools。
 - 对于 spawn_agent, 最终发给 responses.create 的通常是:
 - type

图 4: Codex 内部规格层 (ToolSpec) 与传输定义层 (Wire Tool Definition) 的对比: 内部定义包含 output_schema, 但序列化时会被跳过。⁴

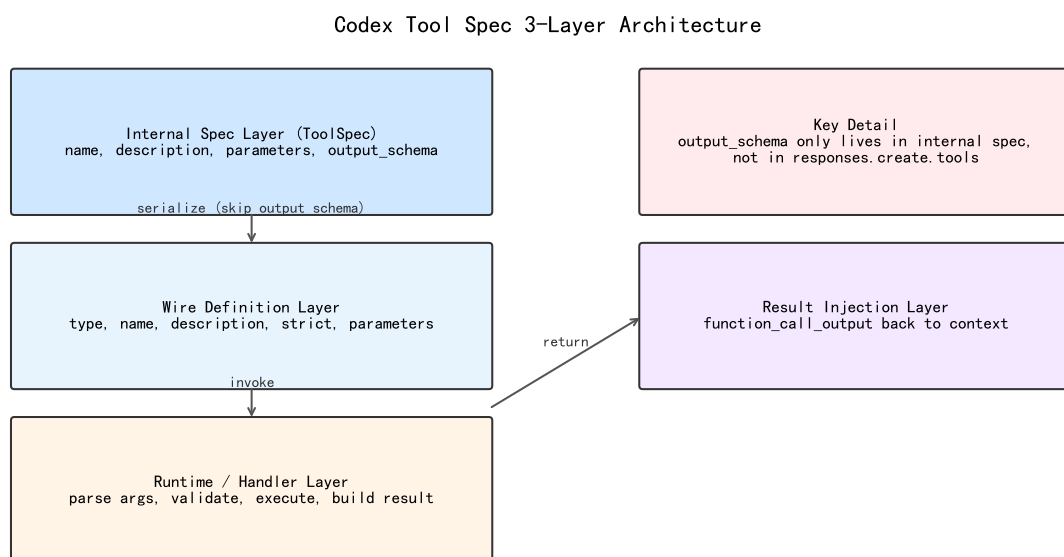


图 5: 手绘三层架构示意图: 内部规格层 -> 传输定义层 -> 执行实现层, 并标注 output_schema 被跳过的关键细节与结果回注路径。⁵

⁴ 视频画面时间区间: 00:09:28-00:09:32。若为裁剪图, 时间区间仍对应原视频画面。

⁵ 视频画面时间区间: 00:09:00-00:11:00。若为裁剪图, 时间区间仍对应原视频画面。

2.5 本章小结

本章介绍了 Codex 工具设计中的三层架构：

1. **Internal spec**: Codex 源码中维护的 ToolSpec 抽象, 包含完整的人参和出参描述(output_schema)。
2. **Wire definition**: 序列化后实际发给 response.create 的 JSON, **不包含** output_schema, 仅约束入参。
3. **Handler / Runtime**: 在本地接收模型的 function call、解析参数、执行工具, 并将结果以 function_call_output 的形式回填到模型上下文中。

通过 spawn_agent 的例子, 我们看到: 模型在调用时并不知道 agent_id 和 nickname, 这些返回值完全由 runtime 在执行后生成并回填。这一机制是所有 Codex 工具的共同基础——无论是改代码的 apply_patch、执行命令的 exec_command, 还是查看图片的 view_image, 其设计都遵循同样的三层分离与回填逻辑。

在下一章中, 我们将把这一框架应用到第一个具体工具 apply_patch 上, 分析 Codex 是如何将代码编辑这一高频操作约束为结构化 DSL 的。

3 apply_patch DSL

在了解了 tool spec 的三层架构之后, 我们来看第一个落地的工具——apply_patch。它是 Codex 中处理代码编辑的核心工具, 也是将高频操作”改代码”约束为结构化 DSL 的典型范例。

3.1 为什么不用 freestyle 写文件, 而是用 patch DSL

代码编辑是 coding agent 最高频的操作之一。一个直观的想法是: 既然模型能生成任意文本, 为什么不直接让它以 freestyle 的方式重写整个文件? Codex 的设计者显然没有走这条路, 而是为 apply_patch 引入了一套受限的 patch DSL (domain-specific language)。

核心设计洞察

Freestyle 重写文件风险极高: 模型可能误删未改动的代码、破坏缩进、引入幻觉修改, 且难以在 handler 层做精确校验。将代码编辑约束为结构化的 patch DSL, 使得每一次增、删、改都可被语法解析、被权限审查、被精确执行。

具体而言, apply_patch 要求模型以结构化的方式声明变更: 添加了哪些行、删除了哪些行、添加了哪个文件、删除了哪个文件。它不是”任意改个文本”, 而是强制使用 begin patch、update file 等关键字驱动的结构化表达。这种设计将”生成自由文本”的问题转化为”生成受约束语法树”的问题, 大幅降低了执行层的不确定性。

常见误解

有同学认为 apply_patch 只是让模型输出一段 diff 文本, 由 runtime 直接丢给 git apply。实际上, Codex 的 patch DSL 是基于 Lark grammar 自定义的语法, 并非标准 unified diff, handler 会自行解析并执行写入, 而不是依赖外部 patch 命令。

3.2 Lark grammar 约束与结构化输出

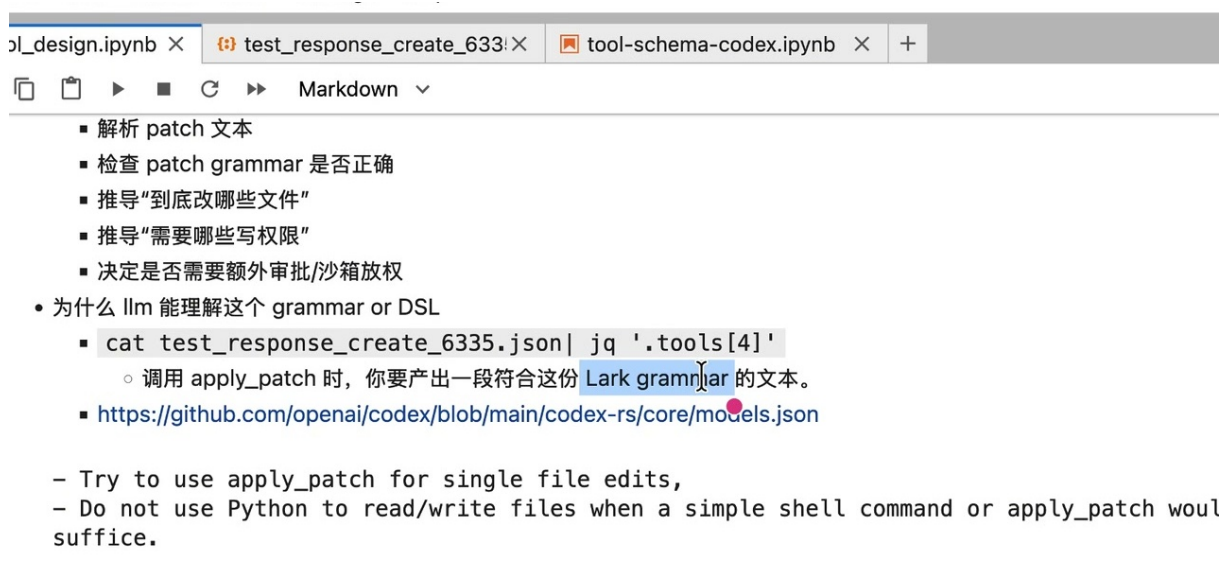
Codex 为 `apply_patch` 定义了一套 Lark grammar。模型输出的 patch 文本必须符合这套语法约束，handler 才能正确解析。常见的顶层关键字包括：

- `begin patch`: 标识一个 patch 块的开始。
- `update file`: 声明对某个已有文件的更新操作。
- `add file`: 声明新增一个文件。
- `remove file`: 声明删除一个文件。

在 `update file` 内部，模型需要精确指出删除的行和新增的行，而不是输出整份重写后的文件内容。这种细粒度的结构化输出有两个好处：一是变更范围清晰，便于 diff 审查；二是 handler 可以在执行前做严格的语法和语义校验。

Lark 是什么

Lark 是一个 Python 编写的解析库，支持 EBNF 风格的 grammar 定义。Codex 将 `apply_patch` 的语法规则用 Lark grammar 描述后，直接嵌入到工具的参数说明或系统提示词中，引导模型生成符合规范的输出。



subagents design

图 6: `apply_patch` 的设计原则：把代码修改变成受限 DSL，handler 先解析 patch 文本、检查 Lark grammar、推导修改文件与权限，再决定是否执行。⁶

3.3 Handler 的执行流程：语法校验 → 推导变更文件 → 权限审批 → 实际写入

当模型发起 `apply_patch` 调用后，handler 并不会立即修改磁盘文件，而是遵循一个分阶段的执行流程：

⁶视频画面时间区间：00:12:02–00:12:06。若为裁剪图，时间区间仍对应原视频画面。

1. **语法校验**: 首先用 Lark grammar 解析模型输出的 patch 文本, 验证其语法是否正确。如果语法错误, handler 会直接返回错误信息, 拒绝执行。
2. **推导变更文件**: 在语法校验通过后, handler 进一步分析 patch 内容, 推导出哪些文件将被修改、新增或删除。
3. **权限审批**: 根据推导出的文件变更列表, handler 检查所需的写权限以及任何额外的审批策略 (例如是否涉及敏感路径、是否超出用户授权范围)。
4. **实际写入**: 只有在以上三步全部通过后, handler 才会将变更真正落盘, 并构造执行结果回填到模型上下文。

这个四步流程体现了 Codex 在”便利”与”安全”之间的权衡: 模型负责生成变更意图, 而 runtime 负责将意图转化为受控、可审查、可回滚的实际操作。

3.4 系统提示词中的使用规范

除了 grammar 和 handler 的硬约束, Codex 还在系统提示词(system instruction)中对 `apply_patch` 的使用方式给出了明确的规范指导。这些规范直接影响模型的工具选择行为:

- **单文件编辑优先使用 `apply_patch`**: 当需要对单个文件进行编辑时, 应调用 `apply_patch`, 而不是用 Python 脚本去读写文件。
- **避免 Python 直接 I/O**: 系统提示词明确告诫模型, 不要用 Python 的 `open()`、`write()` 等原生接口操作文件系统。
- **简单操作走工具通道**: 当简单的 shell 命令或 `apply_patch` 就能满足需求时, 优先选择工具调用, 而不是生成一段 Python 代码来间接完成。

这些规范的存在, 确保了 Codex 在执行代码编辑任务时, 能够始终走受控的 patch DSL 路径, 而不是让模型自由发挥去操作文件系统。

3.5 本章小结

`apply_patch` 是 Codex 通用工具设计中”DSL 化约束”思想的集中体现。本章的核心要点可以总结为以下三条:

1. **安全优先**: 代码编辑被约束为结构化的 patch DSL, 而非 freestyle 重写, 以降低执行风险和幻觉修改。
2. **语法驱动**: Lark grammar 为 `apply_patch` 提供了形式化的语法约束, 关键字如 `begin patch`、`update file` 构成了模型输出的结构化模板。
3. **分阶段执行**: handler 遵循”语法校验 → 推导变更 → 权限审批 → 实际写入”的流程, 确保每一次文件变更都在可控范围内执行。

讲完代码编辑的 DSL 化设计后, 下一章我们将转向命令行交互与并发子系统——先看看 Codex 的 subagent 机制如何通过五个核心工具实现上下文隔离与并发执行。

4 Subagents 系统：并发、隔离与汇总

前置知识

本章延续第 2 章提出的“三层架构”视角（internal spec / wire definition / handler），来分析 Codex 中最复杂的并发子系统——subagents。理解 subagent 的关键不在于模型本身“会分身”，而在于 runtime 如何通过五个核心工具为模型营造出“并发、隔离、可汇总”的执行环境。

4.1 设计动机：上下文隔离、并发执行与 context window 突破

在复杂的代码库分析或多步骤任务中，master agent 面临三个典型瓶颈：

1. **上下文污染**：不同子任务的中间推理、错误尝试、临时结论全部堆积在同一个对话上下文中，容易互相干扰。
2. **串行效率低**：如果所有步骤必须一步一步执行，整体耗时随任务数量线性增长。
3. **context window 限制**：单条对话历史过长时，触发上下文压缩甚至截断，导致关键信息丢失。

Codex 的 subagent 机制正是为了解决上述问题。通过 `spawn_agent` 创建的子代理拥有与 master agent 隔离的上下文空间；多个子代理可以并发运行；最终，master agent 只需等待各子任务的结果并进行汇总（reduce），而不必把每个子任务的完整推理过程都保留在自己的上下文中。这相当于在软件工程中把“一个大单体”拆分为“多个微服务”——每个服务独立演进，最终结果通过明确的接口返回。

核心洞察

Subagent 的价值不仅是“并发提速”，更重要的是通过 **上下文隔离与结果汇总** 来突破 master agent 的 context window 限制。模型不需要看到子任务内部的全部思考链，只需要看到最终的 `function_call_output`。

4.2 五个核心工具：spawn、send_input、wait、close、resume

Codex 在源码中为 subagent 交互定义了五个专用工具，它们共同构成 master agent 与子代理之间的全部通信协议：

- `spawn_agent`：创建一个新的子代理实例。
- `send_input`：向一个已存在的 subagent 发送新的任务指令。
- `wait_for_subagent`：阻塞等待一个或多个 subagent 到达最终状态。
- `close_subagent`：显式关闭/结束一个 subagent。
- `resume_subagent`：恢复一个已经退出或关闭的 subagent，沿原有线程继续工作。

这五个工具的语义非常清晰：`spawn` 是“启动”，`send_input` 是“派发新任务”，`wait` 是“同步等待结果”，`close` 是“终止”，`resume` 是“复活”。master agent 与子代理之间只存在从主到子的单向通信通道，目前接口层面没有中途双向交互的模式。

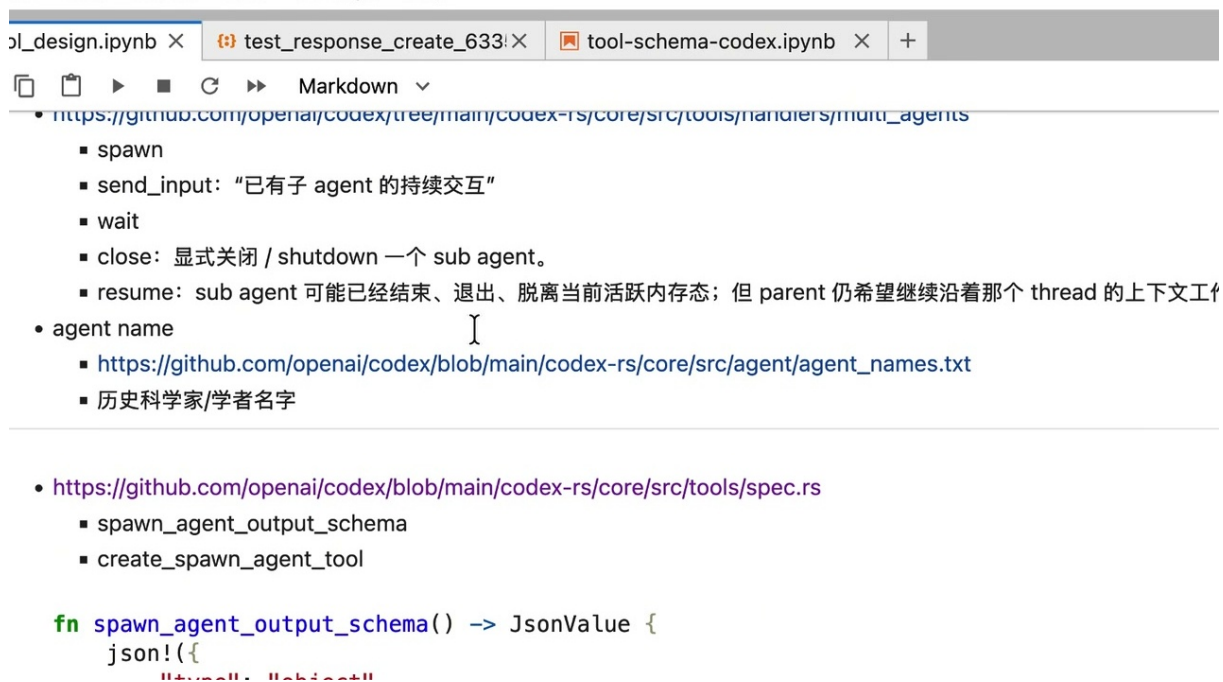


图 7: Subagents 的五个核心工具: spawn、send_input、wait、close、resume, 以及 agent name 的决策逻辑 (从 nickname_candidates 或内置名字池选取)。⁷

4.3 spawn_agent 的返回值机制: agent_id 与 nickname 的回填

spawn_agent 的入参中并不包含 agent_id, 它的 wire definition 只要求模型提供任务描述、系统提示词、输入消息等创建所需的信息。那么, master agent 如何在后续调用中引用这个新创建的子代理呢?

答案仍然回到第 2 章强调的关键机制: 如前所述, **runtime 维护返回值, 并通过 function_call_output 回填到后续上下文。**

当 spawn_agent 的 handler 执行成功后, runtime 会生成一个全局唯一的 agent_id, 并从前端配置或内置名字池中分配一个未被占用的 nickname。这两个字段被包装为 SpawnAgentResult, 最终以 function_call_output 的形式注入 master agent 的后续对话上下文。模型由此“感知”到: 我刚刚创建的子代理, 它的 ID 是什么, 它的展示名称是什么。

⁷ 视频画面时间区间: 00:13:37–00:13:41。若为裁剪图, 时间区间仍对应原视频画面。



图 8: spawn_agent 的返回值示例: runtime 将 {agent_id, nickname} 作为 function_call_output 回填到后续上下文, 使主代理能建立与子代理的关联。⁸

常见误解

有同学会误以为模型自己“记得”了 agent_id, 或者认为 ID 是模型在调用时自行指定的。实际上, 模型对 agent_id 和 nickname 的感知完全依赖于 handler 层的回填。这与 function calling 的一般模式完全一致: 模型只负责发起调用, 返回值的构造与注入由 runtime 完成。

关于 nickname 的分配逻辑: 如果前端配置中提供了专属的 nickname_candidates 列表, runtime 会优先从中选取未被占用的名字; 否则, 就从内置的历史科学家/学者名字池中随机分配。所有实际交互仍然以 agent_id 为准, nickname 仅用于前端展示。

4.4 wait_for_subagent: 状态流订阅而非 sleep

wait_for_subagent 是 subagent 系统中最容易被误解的工具。它的名字里虽然有“wait”, 但它不是一个简单的 sleep 或忙等待。

从实现语义上看, wait_for_subagent 接收一个 agent_id 列表, 然后阻塞当前 master agent 的后续推理, 直到列表中的所有 (或指定) 子代理到达 final state。这更像编程语言中的 join 或 await——它是对子代理 **状态流** 的订阅, 而不是对时间的空转消耗。

核心洞察

wait_for_subagent 的本质是 **状态流订阅**。runtime 会监听指定 subagent 的生命周期事件, 在其到达最终状态时立即唤醒 master agent; 如果超时 (最长约 1 小时), 则返回超时结果, 由 master agent 决定下一步策略 (例如 close 或重新派发)。

⁸视频画面时间区间: 00:14:36-00:14:40。若为裁剪图, 时间区间仍对应原视频画面。

什么时候应该调用 `wait_for_subagent`? 只有当 master agent 的下一步执行**必须依赖**某个 sub-agent 的结果, 且该 subagent 尚未完成时, 才需要触发 wait。如果 master agent 自己还有别的任务可以并行处理, 就无需过早 wait, 让 subagent 在后台继续运行即可。



图 9: `wait_for_subagent` 的 JSON 参数与使用场景: 不是通用 sleep, 而是基于状态订阅等待一个或多个子代理到达 final status。⁹

另外, 即使 master agent 没有显式调用 `wait_for_subagent`, 当某个子代理执行完毕后, runtime 也会自动以 user 身份向 master agent 的上下文发送一条子代理完成消息。这意味着子代理不会“默默失联”, master agent 总能在后续某一轮推理中感知到它的结果。

4.5 触发策略与当前接口的局限

Codex 的 subagent 系统虽然设计精巧, 但在实际使用中有一个显著的触发门槛: 模型**不会**轻易决定调用 `spawn_agent`。原因在于工具 description 中写了非常强的约束指令:

- 只有当用户**显式要求**并发执行或分派任务时, 才创建 subagent。
- 例外情况: 如果任务是对一个代码库进行**深度、全面的研究或调查** (deep comprehensive research), 则无需用户许可即可创建 subagent。

这两条强指令极大地抑制了 subagent 的自发触发。因此, 如果你想在 Codex 中体验 subagent 的并发流程, 最好在用户查询中明确写出“请使用 subagent 并发/分派地完成这个复杂任务”。

⁹视频画面时间区间: 00:18:24–00:18:28。若为裁剪图, 时间区间仍对应原视频画面。

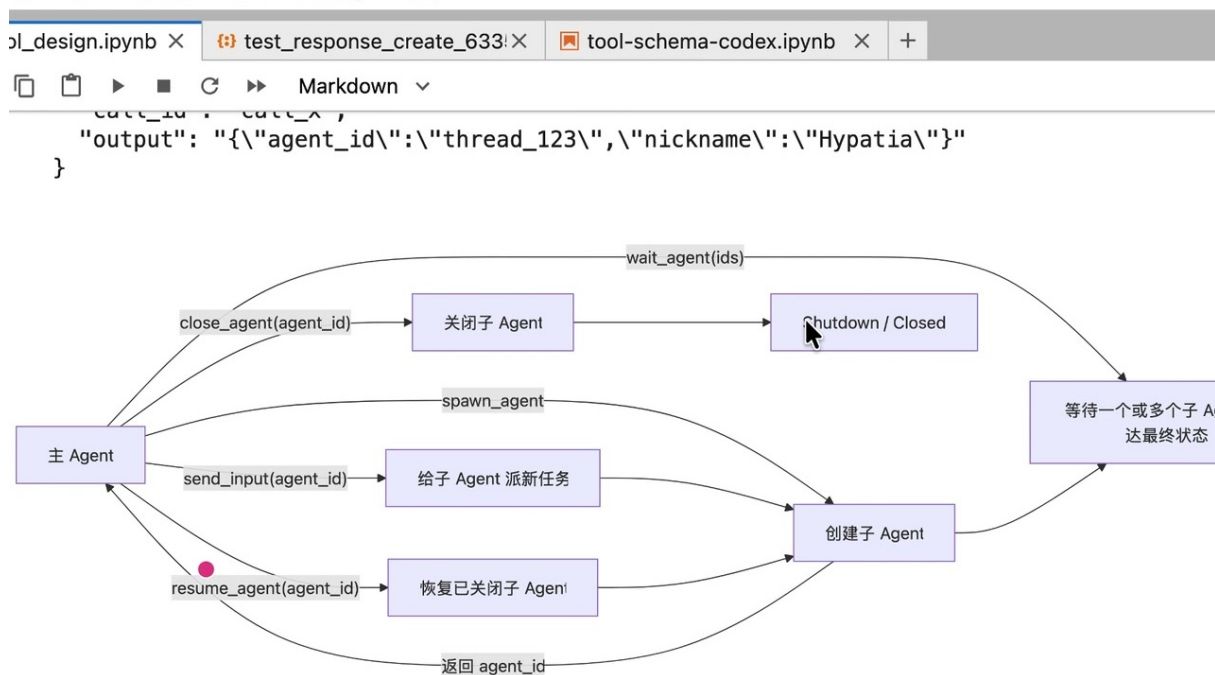


图 10: 主代理与子代理交互架构图: 展示 `spawn_agent`、`send_input`、`wait_for_subagent`、`close_subagent`、`resume_subagent` 的完整调用关系与状态流转。¹⁰

当前接口的局限

目前的 subagent 接口存在一个明显限制: 子代理只能在结束时输出结果, 无法与 master agent 进行中途双向交互。master agent 可以向子代理单向发送指令 (`send_input`), 但子代理在执行过程中不能主动向 master agent 汇报进度、请求澄清或交换中间状态。这种“单向、批处理”式的通信模式, 限制了更复杂的协作场景。

4.6 本章小结

本章系统梳理了 Codex subagent 并发子系统的设计思想与五个核心工具:

1. **设计动机**: 通过上下文隔离、并发执行与结果汇总, 解决 master agent 的上下文污染和 context window 瓶颈。
2. **五个工具**: `spawn_agent`、`send_input`、`wait_for_subagent`、`close_subagent`、`resume_subagent` 构成完整的生命周期管理协议。
3. **返回值回填**: `agent_id` 与 `nickname` 由 runtime 在 handler 层生成, 并通过 `function_call_output` 注入上下文, 模型由此建立对子代理的引用。
4. **wait 的语义**: `wait_for_subagent` 是对状态流的订阅 (join/await), 而非 sleep; 只有在必须依赖结果时才应触发。
5. **触发策略与局限**: 工具 description 中的强指令严格约束了 subagent 的创建条件; 当前接口不支持子代理与 master agent 的中途双向交互。

¹⁰视频画面时间区间: 00:16:08–00:16:12。若为裁剪图, 时间区间仍对应原视频画面。

讲完这套复杂的并发调度机制后，下一章我们将进入最后一个重点工具——`view_image`，探讨多模态输入是如何被 agent 主动调用的。

5 `view_image`: Agentic Vision 的本地桥接

在讲完 subagent 的并发调度之后，我们把目光转向 Codex 通用工具设计中的最后一个重点——多模态输入的桥接。

多模态大模型理论上可以直接消费 base64 编码的图片，但在一个 coding agent system 中，图像不会自动出现在模型面前；它必须由模型主动决定看什么、由 runtime 负责把本地图片准备好并注入上下文。这正是 `view_image` 存在的意义。

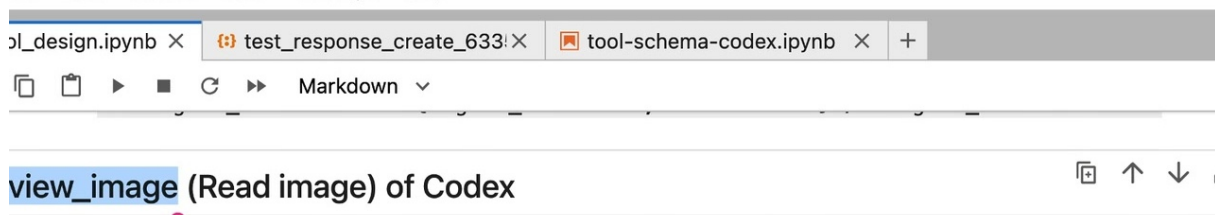
5.1 为什么需要专门的 `view_image` 工具，而非直接塞 base64

一个自然的疑问是：既然底层模型支持图像输入，为什么不在请求体里直接塞入 base64 图片，反而要设计一个专门的工具？

常见误解

有人误以为多模态能力意味着“只要图片在本地，模型就能自动看到”。实际上，模型对视觉输入的感知完全取决于谁决定把哪张图片放进上下文。如果由外部系统无条件塞入所有图片，模型就失去了对视觉注意力的自主控制。

在 coding agent 的语境下，`view_image` 是 *agentic vision* 能力的来源。基础模型必须是 VLM (Vision-Language Model)，能够接受 `input_image` 类型的消息；但更重要的是，准备图像输入 (*prepare image input*) 这一步必须由 *agentic runtime* 来完成。模型在推理过程中自主判断“我现在需要查看某张具体图片”，于是发起 `view_image` 调用；runtime 接到调用后，再去读取、编码并回注。这种“模型决策 + runtime 执行”的分工，与前文讲过的 `apply_patch`、`spawn_agent` 等工具遵循的是同一套三层架构逻辑。



- 多模态 agent 能力的来源；（当然首先 base model 是一个vlm，能接受image input）
- 为什么需要一个单独的 tool/function 是 agent 需要自主维护看什么图片，放到 context 中；
 - “prepare image input”这一步必须由 runtime 做，而不是模型自己做

常见直接调 API，如果你的应用自己已经拿到了图片数据或 URL（“客户端自己已经有图片”），最常见的就是发一条多模态 user message：

```
{
  "type": "message",
  "role": "user",
  "content": [
    { "type": "input_text", "text": "这张图里有什么?" },
    { "type": "input_image", "image_url": "data:image/png;base64,..." }
  ]
}
```

图 11: view_image 的本质：Codex 的本地图像桥接工具。runtime 负责把本地图片读取、缩放并转成 base64 的 input_image，再注入到后续 responses.create.input 中。¹¹

5.2 runtime 职责：路径解析 → 读取文件 → base64 编码 → 注入上下文

view_image 的 handler 层工作流程非常清晰，可以概括为四个连续步骤：

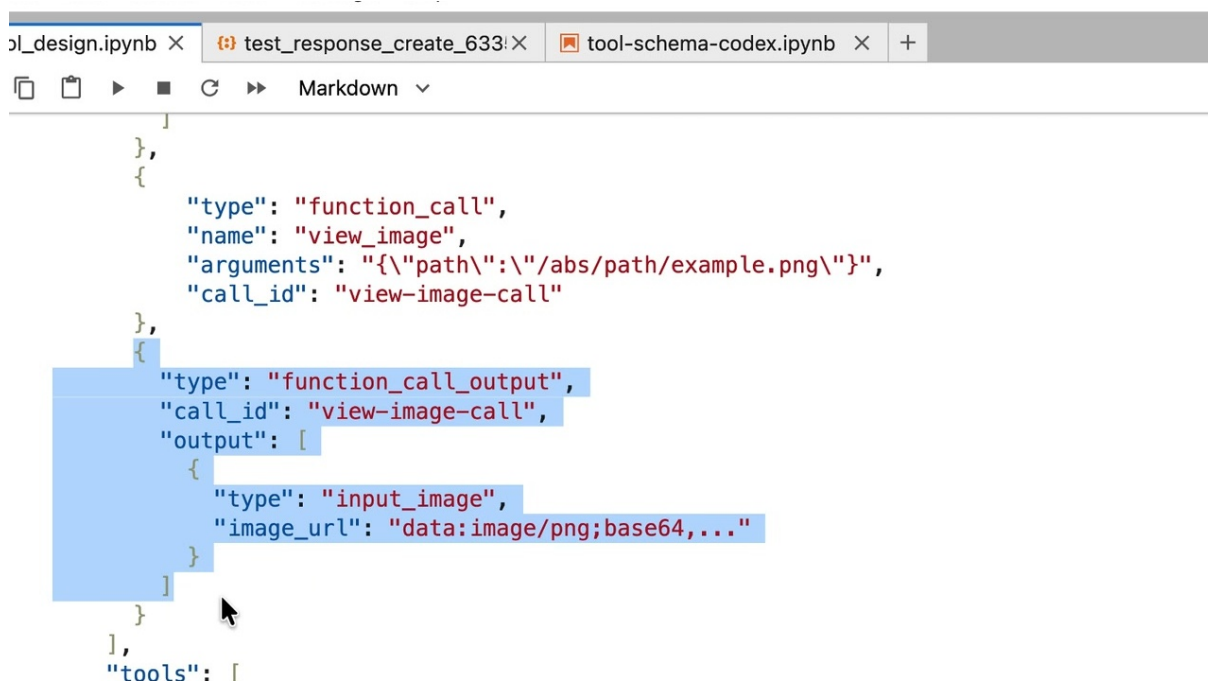
1. **路径解析**：从模型传来的参数中提取图片的完整本地路径；
2. **读取文件**：打开对应图片，必要时进行缩放或格式转换；
3. **base64 编码**：将图像二进制数据编码为 base64 字符串；
4. **注入上下文**：把编码后的图像包装为 function_call_output 中的 input_image 内容，插入到后续 responses.create 的输入消息列表中。

核心洞察：返回值不是文本，而是图像本身

与其他工具返回纯文本或 JSON 不同，view_image 的 function_call_output 直接携带 input_image 类型的消息。runtime 回注的不再是“图片里有什么”的文字描述，而是 data:image/png;base64,... 这样的图像数据本身。这意味着模型在下一轮推理中，可以像看到用户直接上传的图片一样“看见”这张本地图片。

例如，当用户说“请读取这张照片”并提供了一个绝对路径时，模型会产生一次 view_image 的 function call；Codex runtime 执行该调用，将图像编码为 base64，然后以 function_call_output 的形式把图像消息塞入上下文。

¹¹ 视频画面时间区间：00:20:16–00:20:20。



```
    },
    {
      "type": "function_call",
      "name": "view_image",
      "arguments": "{\"path\": \"./abs/path/example.png\"}",
      "call_id": "view-image-call"
    },
    {
      "type": "function_call_output",
      "call_id": "view-image-call",
      "output": [
        {
          "type": "input_image",
          "image_url": "data:image/png;base64,..."
        }
      ]
    }
  ],
  "tools": [
```

图 12: view_image 的 function_call_output 示例：输出类型为 input_image, image_url 为 data:image/png;base64,..., 直接作为模型可见的多模态输入。¹²

5.3 使用条件：完整路径 + 历史上未查看过

为了控制 token 消耗并避免重复读取，view_image 的 tool description 中对调用条件有明确约束：

工具描述摘要

查看一个本地图片，从文件系统中读取。**当且仅当**用户给了一个完整的文件路径，**且**该图片在历史上没有出现在对话上下文中时，才调用此工具。

这两个条件的用意十分明确：

- **完整路径**：防止模型对模糊文件名进行猜测，确保 runtime 能够准确定位到文件系统上的唯一资源；
- **历史上未查看过**：如果同一张图片已经在之前的对话中作为 input_image 被注入过上下文，那么模型可以直接复用已有的视觉信息，无需再次调用 view_image 浪费 token。

这种复用逻辑与 Codex 整体的 context compression 策略是一致的：能省则省，只在必要时扩充上下文。

¹²视频画面时间区间：00:21:36–00:21:40。



`view_image` 是 Codex 的本地图像桥接工具。它在 tool definition 层向模型暴露“查看本地图片”的能力，在 runtime 层负责完成析、文件读取、缩放/保真策略和 data URL 转换，最终并不是返回普通文本或 JSON，而是把图片作为 `function_call_output` 中 `input_image` 内容项注入后续 `responses.create.input`。因此，`view_image` 的本质不是“让模型看图”，而是“把本地文件系统里全地转成模型真正能看到的图”。

```
{
  "type": "function_call_output",
  "call_id": "call_view_image_x",
  "output": [
    {
      "type": "input_image",
      "image_url": "data:image/png;base64,...",
      "detail": "original"
    }
  ]
}
```

图 13: `view_image` 的 tool description 摘要：仅当用户提供了完整文件路径且图片未出现在对话上下文中时才调用，是本地文件系统到模型视觉输入的安全桥接。¹³

5.4 多模态能力与 coding agent 的结合

`view_image` 的设计体现了 coding agent 对多模态能力的独特需求：不是让人类提前把所有可能的截图、设计稿、运行结果图都喂给模型，而是让模型在解决问题的过程中按需索取。

这种按需索取的模式带来了几个实际好处：

1. **上下文精简**：模型只看它认为相关的图片，避免无关图像挤占宝贵的 context window；
2. **推理可追溯**：每一次看图都对应一次明确的 `view_image` function call，便于事后 review 和调试；
3. **与代码工具协同**：模型可以先运行 `exec_command` 生成图表或截图，再通过 `view_image` 查看结果，形成“执行—观察—再编辑”的闭环。

Agentic Vision 的本质

Agentic Vision 的关键不在于“模型能不能看懂图”，而在于模型能不能自主决定看什么图。`view_image` 就是这个决策权的抓手：它把“视觉注意力”从外部系统手中交还给了模型本身，同时由 runtime 承担繁重的本地文件桥接工作。

5.5 本章小结

本章围绕 `view_image` 展开了四方面讨论：

- **设计动机**：在 coding agent 中，多模态输入不能由外部系统无条件塞入，而必须通过专用工具让模型自主决定看什么图片；

¹³ 视频画面时间区间：00:22:00–00:22:04。

- **Runtime 职责**：路径解析、文件读取、base64 编码、以 `function_call_output` 形式回注图像消息，四步缺一不可；
- **调用约束**：完整路径 + 历史上未查看过，既保证安全性又避免重复消耗 token；
- **系统价值**：`view_image` 让 coding agent 具备了按需索取视觉信息的能力，与 `apply_patch`、`exec_command`、`subagents` 等工具共同构成了完整的 agent 工作闭环。

至此，Codex 通用工具设计中的核心工具——编辑、执行、并发、视觉——均已梳理完毕，它们共同构成了 12 个预定义工具的最小必要集合。

6 总结与延伸

6.1 演讲者的核心收束

在视频的尾声，演讲者再次强调了贯穿全场的核心主张：**不要把 Codex 当成黑盒**。当我们看到 agent 的表现不符合预期时，第一反应不应该是盲目地重试或更换 prompt，而是要去 *review* 它的 tool use 过程——看看模型调用了哪些工具、参数是否合理、runtime 是否正确执行并回填了结果。这种“白盒”审视的能力，是区分“会用 agent”和“真正理解 agent”的关键。

演讲者还指出，Codex 的 12 个预定义工具并非随意堆砌，而是围绕“编辑、执行、并发、感知、交互”五大类需求设计的最小必要集合。理解了这些工具的 spec、wire definition 和 handler 三层分离，就理解了 Codex 如何将底层模型的通用能力转化为可控、可追踪、可调试的 agentic 行为。

6.2 结构化提炼：五条核心洞察

基于对本节课程的系统梳理，我们可以将 Codex 的通用工具设计压缩为以下五条核心洞察：

核心洞察

1. **三层架构是理解一切的基础**：internal spec 描述工具全貌（含 output_schema），wire definition 仅约束模型可见的入参，handler/runtime 负责执行并将返回值回填上下文。这一分层让”模型不知道返回值长什么样”成为可能，也是 subagent、view_image 等复杂工具得以简洁定义的前提。
2. **DSL 化是对高频操作的安全约束**：apply_patch 将代码编辑从自由文本重写约束为受 Lark grammar 管控的 patch DSL，既节省了上下文 token，又将高风险操作纳入可验证、可审批的执行链路。
3. **命令行是 agent 与外部世界交互的通用接口**：exec_command 和 write_stdin 让模型能够利用操作系统提供的丰富 CLI 生态，把”生成代码”和”执行代码”两个环节打通，实现真正的端到端自动化。
4. **并发不是并行计算，而是上下文管理**：subagent 系统的核心价值不在于利用多核算力，而在于通过上下文隔离和结果汇总来突破 master agent 的 context window 限制。wait_for_subagent 的 join/await 语义也体现了这一点。
5. **多模态需要 agentic 决策权**：view_image 的设计表明，coding agent 中的视觉能力不只是”模型能不能看懂图”，更重要的是”模型能不能自主决定看什么图”。runtime 桥接本地文件系统与模型视觉输入，而决策权始终保留在模型手中。

6.3 从 tool design 反推：如何 review 和引导 agent

理解了 Codex 的工具设计之后，我们可以把这套认知迁移到实际工程中去 review 和引导任何 coding agent 的交付过程。一个实用的 checklist 包括：

- **检查 tool description**：模型是否因为 description 的措辞过于宽泛或过于严格而误触发/漏触发某个工具？
- **检查 handler 执行边界**：runtime 是否对高风险操作（如写文件、执行命令）设置了足够的校验和权限审批？
- **检查返回值回填时序**：模型在某一轮推理中”似乎不知道”某个工具的结果，是否因为回填消息没有正确插入到后续请求中？
- **检查上下文压缩策略**：subagent 的引入是否合理？是否存在可以通过子代理隔离来减少 master context 冗余的场景？
- **检查多模态调用条件**：模型调用 view_image 时是否满足”完整路径 + 历史上未查看过”的约束？

6.4 拓展阅读

本门课程的分析基于 OpenAI Codex 的开源实现与官方文档，以下是两个重要的参考资源：

- **课程配套 Notebook**：https://github.com/wdkns/modern_genai_bilibili/blob/main/agents/context-engineering/general_tool_design.ipynb

- 示例请求体 JSON:https://github.com/wdkns/modern_genai_bilibili/blob/main/agents/context-engineering/files/test_response_create_6335.json

对 Codex 工具设计感兴趣的读者，还可以进一步关注 OpenAI 官方对 Responses API 中 function calling 规范的更新，以及 Codex CLI 开源仓库中 `spec.rs` 和各个 handler 的实现细节。唯有持续地“拆黑盒”，才能在 agentic 编程的浪潮中保持清醒的工程判断力。